Predictions & Prediction Plots

June 2025

What Does It* All Mean?

*Especially all those $\hat{\beta}$s...

Task #1:

A. Write model equation

B. In words, how does [outcome] depend on [predictor]?

```
##
## Call:
## lm(formula = weight_change ~ diet + age + sex + first_diet, data = upf_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.4125 -0.3315 -0.1953  0.4192  3.0771
##
## Coefficients:
##                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)           -0.144302   0.728740  -0.198  0.84378
## dietUltra-processed    0.778908   0.342445   2.275  0.02693 *
## dietUnprocessed       -1.066615   0.342445  -3.115  0.00295 **
## age                   -0.003419   0.020212  -0.169  0.86632
## sexMale                0.455869   0.285391   1.597  0.11602
## first_dietUnprocessed  0.046063   0.288712   0.160  0.87383
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.083 on 54 degrees of freedom
## Multiple R-squared:  0.3712,    Adjusted R-squared:  0.3129
## F-statistic: 6.374 on 5 and 54 DF,  p-value: 0.0001015
```

Task #2

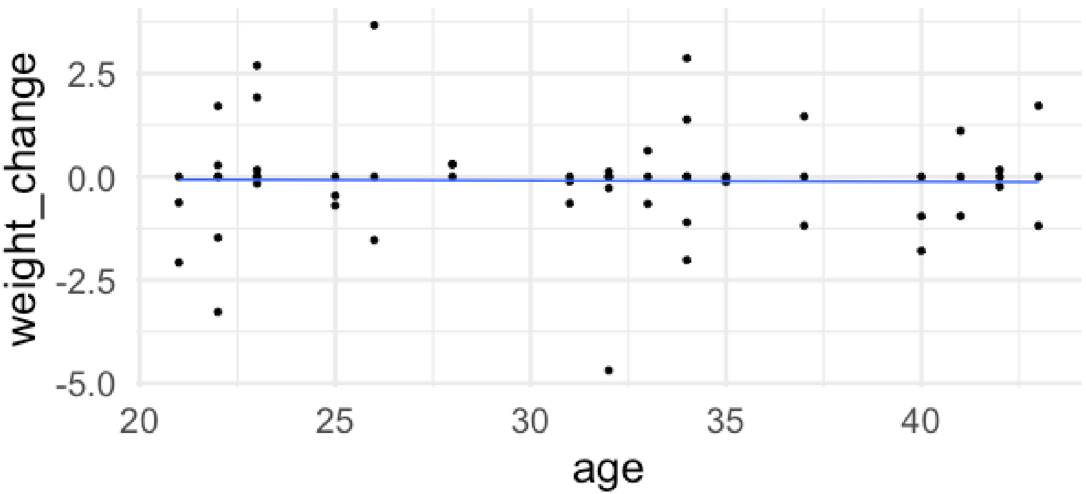
A. Write model equation

B. In words, how does [outcome] depend on [predictor]?

```
##  Family: nbinom2  ( log )
## Formula:
## cancer_cases ~ point_cancer_risk + nonpoint_cancer_risk + percent_poverty +
##     percent_smokers + percent_construction + percent_manufacturing +
##     offset(log_population)
## Data: cancer_data
##
##      AIC       BIC    logLik -2*log(L)  df.resid
##   2475.1    2503.0   -1229.6    2459.1       234
##
##
## Dispersion parameter for nbinom2 family (): 29.4
##
## Conditional model:
##                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)           -3.149483   0.133632 -23.568  < 2e-16 ***
## point_cancer_risk      14.476734  10.046512   1.441  0.14959
## nonpoint_cancer_risk  -14.647271   5.873331  -2.494  0.01264 *
## percent_poverty        -0.276068   0.261552  -1.055  0.29120
## percent_smokers         0.013126   0.005029   2.610  0.00906 **
## percent_construction   -0.163544   0.604979  -0.270  0.78691
## percent_manufacturing   0.101720   0.578105   0.176  0.86033
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Why Not This?

```
gf_point(weight_change ~ age,
         data = upf_data) |>
  gf_lm()
```



Which to Present?

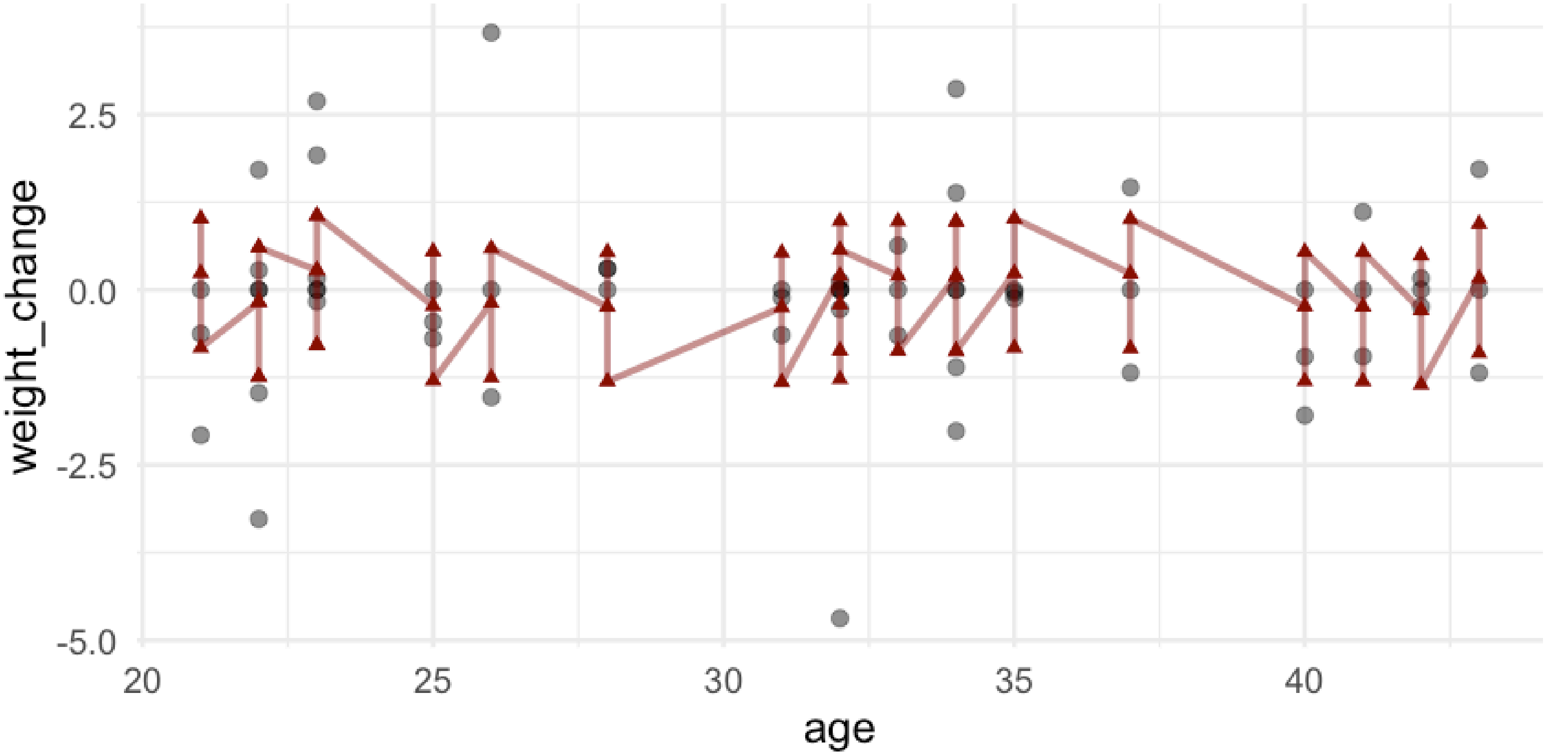
```
coef(lm(weight_change ~ age,
           data = upf_data))
```

```
##  (Intercept)          age
## -0.016614183 -0.002541294
```

```
coef(lm(weight_change ~ age +
          diet + sex +
          first_diet,
           data = upf_data))
```

```
##           (Intercept)                     age   dietUltra-processed
##          -0.144302328            -0.003418705           0.778907692
##       dietUnprocessed                 sexMale first_dietUnprocessed
##          -1.066615385             0.455868929           0.046062892
```

Adding "the line" to "the plot"



So…
what could we do?

Model Predictions...

For *fake* data???

```
library(ggeffects)
upf_predictions <-
  predict_response(upf_model,
                   terms = "age")
```

Model Predictions...

For *fake* data???

```
upf_predictions
```

```
## # Predicted values of weight_change
##
## age | Predicted |      95% CI
## ----------------------------
##  21 |      0.29 | -0.47, 1.04
##  23 |      0.28 | -0.44, 1.00
##  26 |      0.27 | -0.41, 0.95
##  31 |      0.25 | -0.41, 0.91
##  32 |      0.25 | -0.41, 0.91
##  34 |      0.24 | -0.43, 0.92
##  37 |      0.23 | -0.48, 0.94
##  43 |      0.21 | -0.63, 1.05
##
## Adjusted for:
## *       diet =    Baseline
## *        sex =        Male
## * first_diet = Unprocessed
```

```
##
## Not all rows are shown in the output. Use `print(..., n = Inf)` to show
##   all rows.
```

See "fixed" value(s)

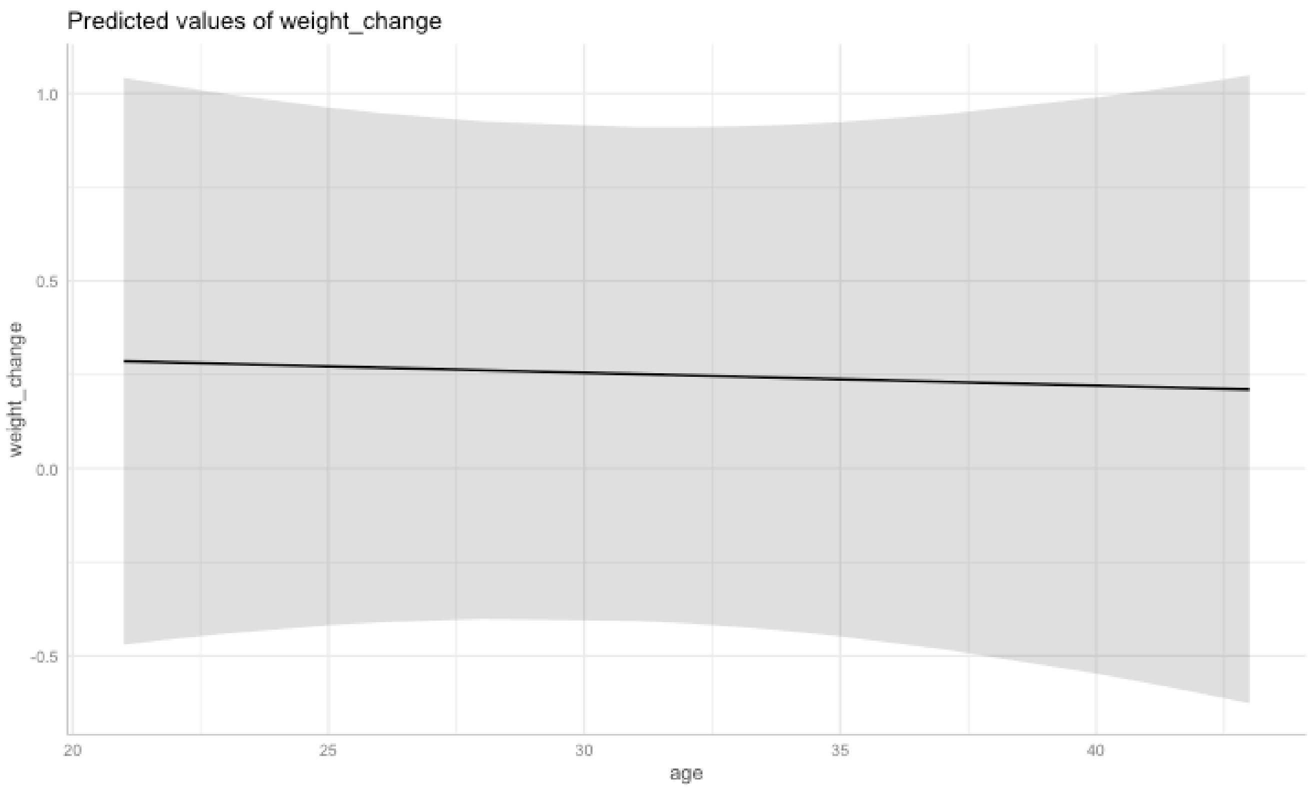
```r
predict_response(
  upf_model,
  terms = "age",
  condition = c(diet = "Ultra-processed"))
```

```
## # Predicted values of weight_change
##
## age | Predicted |     95% CI
## ----------------------------
##  21 |      1.06 | 0.31, 1.82
##  23 |      1.06 | 0.34, 1.78
##  26 |      1.05 | 0.37, 1.73
##  31 |      1.03 | 0.37, 1.69
##  32 |      1.03 | 0.37, 1.69
##  34 |      1.02 | 0.35, 1.70
##  37 |      1.01 | 0.30, 1.72
##  43 |      0.99 | 0.15, 1.83
##
## Adjusted for:
## *        sex =        Male
## * first_diet = Unprocessed
```

```
##
## Not all rows are shown in the output. Use `print(..., n = Inf)` to show
##   all rows.
```

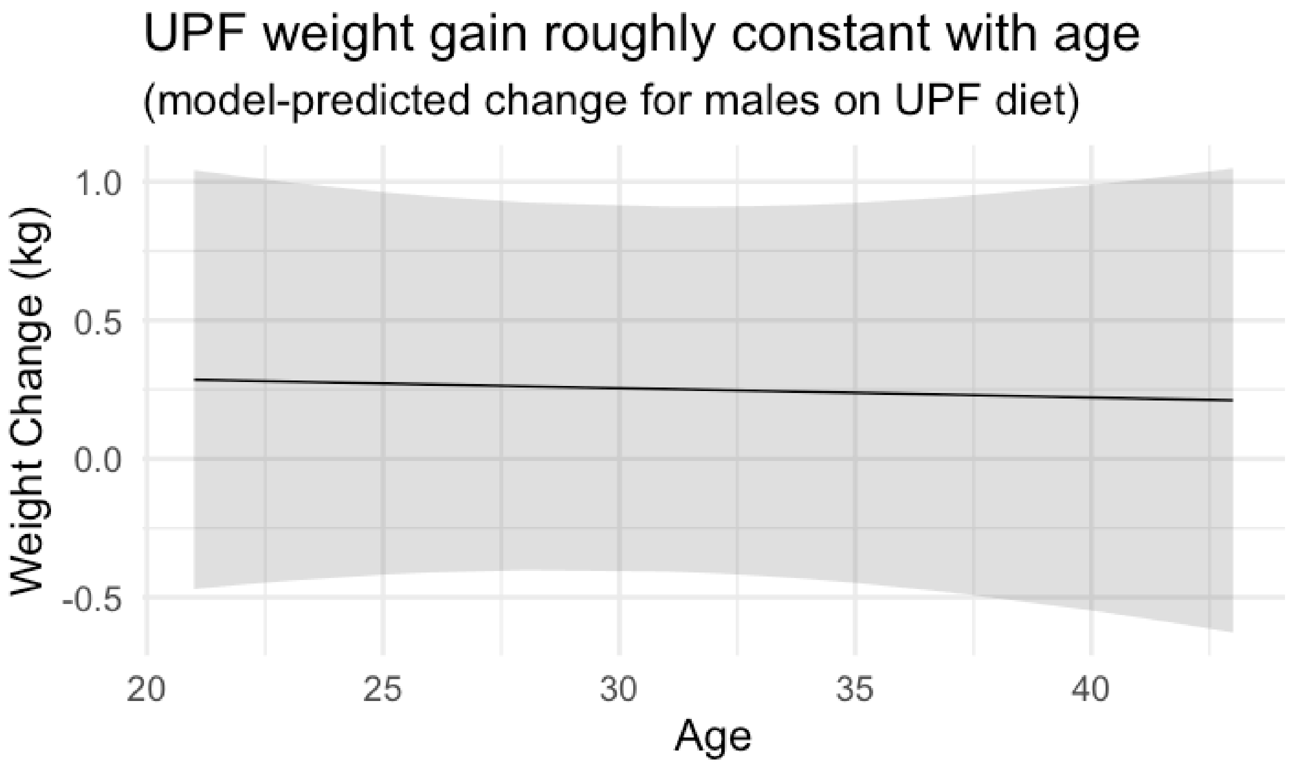
Plot it!

```
plot(upf_predictions)
```



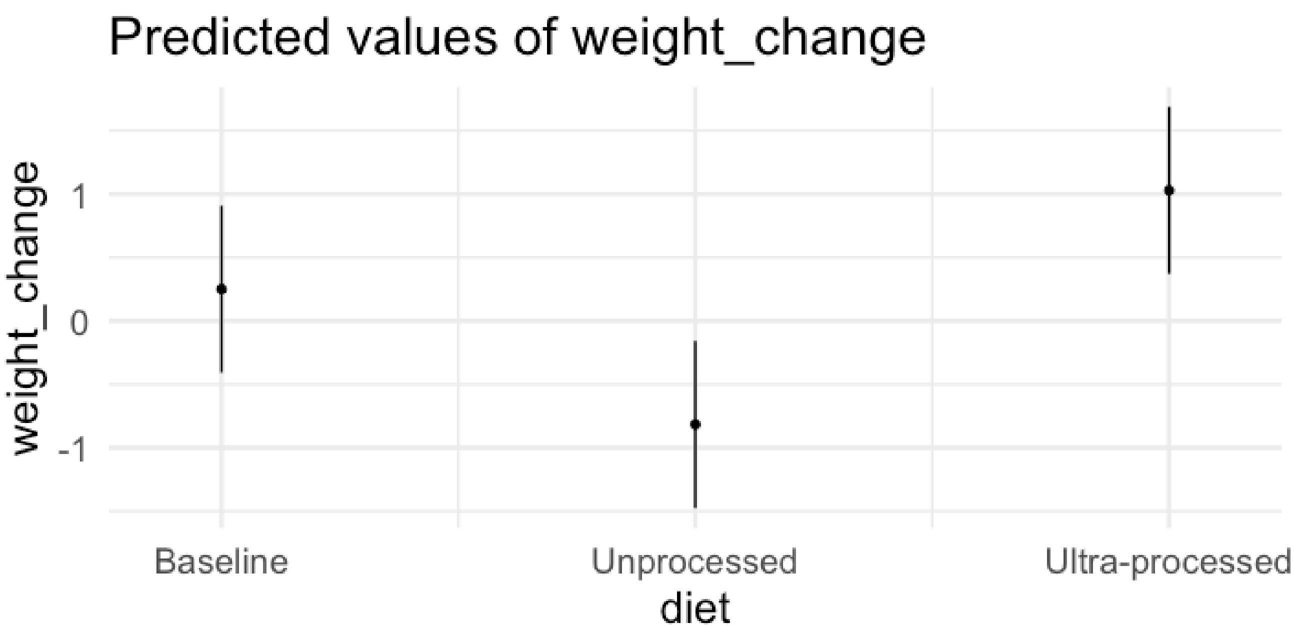
Small Customizations

```
plot(upf_predictions) |>
  gf_labs(x = "Age", y = "Weight Change (kg)",
          title = "UPF weight gain roughly constant with age",
          subtitle = "(model-predicted change for males on UPF diet)")
```



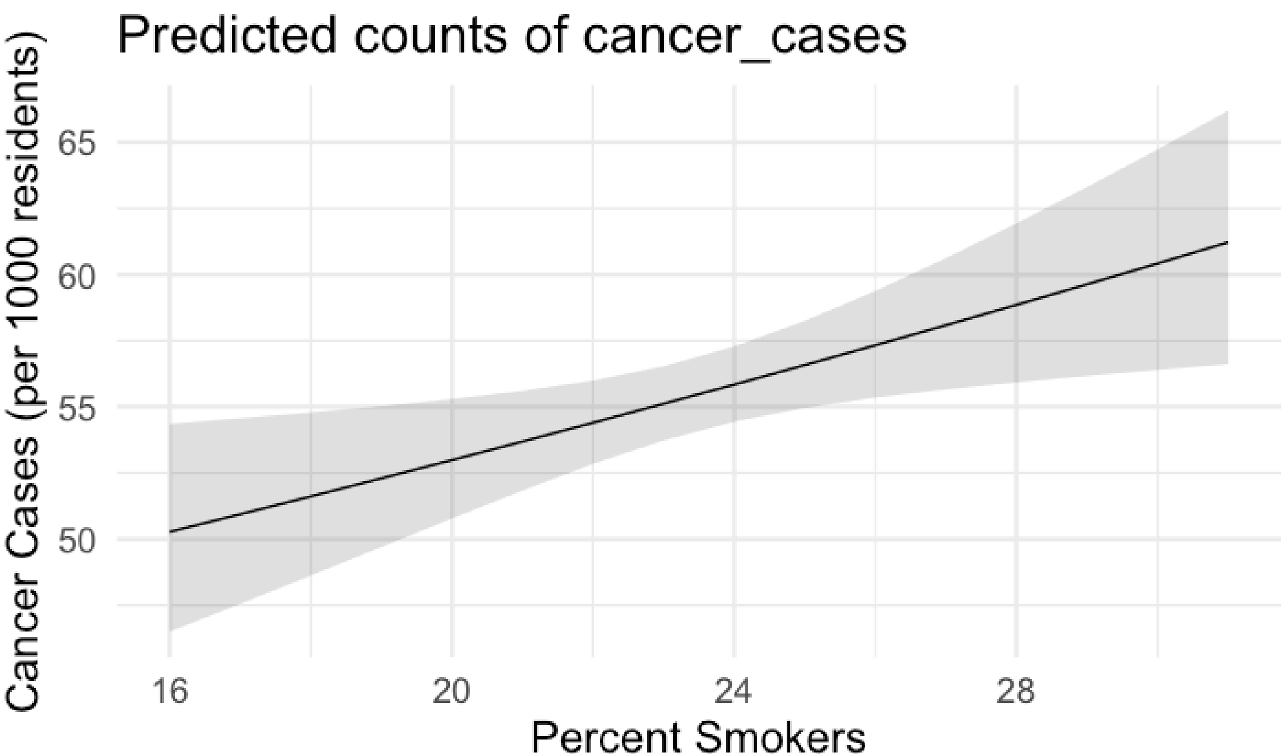
Categorical Too

```r
predict_response(upf_model,
                 terms = "diet",
                 verbose = FALSE) |>
  plot(use_theme = FALSE)
```



When this is *really* useful: GLMs

```
predict_response(cancer_model,
                 terms = "percent_smokers",
                 condition = c(log_population = log(1000)),
                 verbose = FALSE)|>
  plot() |>
  gf_labs(x = "Percent Smokers",
          y = "Cancer Cases (per 1000 residents)")
```



Your Turn

1. Choose a model you fit before

2. Make a prediction plot!

- Note the *fixed values* used.
- What do you learn from the plot?

3. What happens if you use >1 `terms`?

- Code will look like: `predict_response( ... , terms = c("age", "diet"))`

4. What happens if you swap the order of the `terms` being plotted?